

SimilCode: A Web-Based System for Multidimensional Source Code Similarity Detection and Algorithmic Efficiency Analysis Using Generative Artificial Intelligence

Gleiston C. Guerrero-Ulloa¹, Rafael A. Navas-Rivera¹, Efraín-Díaz Macías¹, Carlos Rodríguez Domínguez², and Miguel J. Hornos*²

¹Faculty of Computer Sciences, Universidad Técnica Estatal de Quevedo, Quevedo 120501, Ecuador

²Software Engineering Department, University of Granada, Granada 18014, Spain

Abstract

Background: Source code similarity detection and algorithmic efficiency assessment are persistent challenges in computer science education, particularly in resource-constrained institutions where instructors face overload reviewing student submissions and existing tools rely on syntactic comparison alone. Recent advances in Large Language Models (LLMs) offer semantic-aware analysis, yet empirical evidence comparing commercial LLMs for this dual task is scarce. **Methods:** We developed SimilCode, a web application integrating LLM-based multidimensional similarity detection with automated Big O complexity analysis. We conducted a controlled experiment evaluating four state-of-the-art LLMs (Claude Opus 4.1, GPT-5, Gemini 2.5 Pro, DeepSeek-V3) over 120 curated code pairs in C# and Java spanning four similarity categories. Performance was assessed across precision, response time, and justification quality using standardized prompts engineered with Chain-of-Thought and Chain-of-Verification techniques. Statistical analysis employed one-way ANOVA and Kruskal-Wallis tests ($\alpha = 0.05$). The system was subsequently validated with five expert instructors using a Likert-scale instrument. **Results:** All four models achieved equivalent classification precision (98.3%, $p = 1.000$). Significant differences emerged in response time ($F = 30.89$, $p < 0.001$), with Claude Opus 4.1 fastest (19.08 s) and Gemini 2.5 Pro slowest (25.73 s), and in justification quality (Kruskal-Wallis $H = 11.50$, $p = 0.009$), led by Claude Opus 4.1 (4.96/5). Expert validation yielded a global mean of 4.60/5.00 across usefulness, accuracy, and interpretability dimensions. **Conclusions:** Modern commercial LLMs have reached precision parity for code similarity detection; operational criteria (latency, explanation quality) now drive model selection. SimilCode contributes a unified workflow integrating five-dimensional semantic similarity with Big O efficiency analysis, addressing a documented gap in academic code-evaluation tooling.

Keywords: source code similarity; generative artificial intelligence; large language models; algorithmic efficiency; Big O notation; prompt engineering; programming education; academic integrity.

1 Introduction

Source code similarity detection is a foundational task in software engineering with applications spanning plagiarism identification, clone detection, code recommendation, vulnerability discovery, and quality assurance [1–3]. In higher education, it is particularly critical: programming courses generate large volumes of student submissions whose manual review imposes substantial cognitive and temporal load on instructors [4, 5]. Compounding this burden, novice programmers frequently produce algorithmically inefficient solutions whose evaluation requires complementary, time-consuming complexity analysis [6, 7]. Institutions in low-resource settings face an addi-

tional constraint: commercial similarity-detection tools (e.g., MOSS, JPlag, Codequiry) offer limited semantic reasoning and incur recurrent licensing costs that strain academic budgets [8, 9].

The literature documents two parallel research streams that, until recently, have evolved independently. The first concerns syntactic and semi-semantic similarity detection using token-based comparison, abstract syntax trees, and program-dependency graphs [2, 10, 11]. While these techniques achieve high precision on canonical benchmarks, they remain brittle under simple semantic-preserving transformations and provide no actionable feedback to learners [1, 5]. The second stream investigates Large Language Models (LLMs) as program-comprehension engines, with

applications in code generation [12, 13], automated review [11], and educational assistance [14–16]. LLMs exhibit emergent semantic reasoning over code, enabling cross-implementation comparison and natural-language justifications. However, their systematic benchmarking for similarity detection — and crucially, for the joint task of similarity *plus* efficiency analysis — has not been reported.

A specific gap therefore exists. No published evidence compares the leading commercial LLMs (Claude, GPT, Gemini, DeepSeek) under a unified protocol for source code similarity, and no system has integrated LLM-based multidimensional similarity with automated Big O efficiency analysis in a single educational tool. This gap is consequential for three reasons. First, instructors need empirical guidance to choose among LLM providers given their differing latency, cost, and explanation quality. Second, the educational value of similarity detection is amplified when paired with feedback on algorithmic efficiency — a dimension absent in current plagiarism tools. Third, prompt-engineering techniques such as Chain-of-Thought (CoT) [17] and Chain-of-Verification (CoVe) [18] have not been operationalized for structured code-comparison tasks at the dimensional granularity required by educators.

This study addresses the gap with three contributions. First, we present a controlled comparative evaluation of four state-of-the-art commercial LLMs over 120 curated code pairs in C# and Java, measuring precision, response time, and justification quality through a standardized CoT/CoVe-based prompt. Second, we design and implement *SimilCode*, a web application that integrates five-dimensional similarity detection (lexical, structural, stylistic, functional, syntactic) with automated Big O complexity analysis in a unified workflow. Third, we validate the system with expert instructors and report an integrated discussion of latency–quality trade-offs that informs LLM selection for educational analytics.

The remainder of this article is organized as follows. Section 2 surveys related work. Section 3 details the experimental and engineering methodology. Section 4 reports results. Section 5 discusses implications and limitations. Section 6 concludes.

2 Related Work

This section surveys four research strands that converge on the present contribution: code sim-

ilarity detection, generative AI applied to software engineering, automated algorithmic efficiency analysis, and prompt engineering for structured tasks. For each strand we (i) describe representative works, (ii) highlight commonalities, (iii) contrast their methodological choices, and (iv) identify residual deficiencies. The section closes with a synthesis (Section 2.5) that articulates the additional advantages of *SimilCode* over the reviewed body of work.

2.1 Code Similarity Detection

Classical approaches to similarity detection rely on lexical comparison (token-based), structural representations such as abstract syntax trees (AST), or program-dependency graphs (PDG) [1, 2]. Recent work has migrated towards machine-learning-based representations: Li et al. [10] reported a graph-neural-network model achieving precision above 90% on industrial datasets, while Zhang and Saber [3] provide a systematic review showing that learned representations consistently outperform purely syntactic baselines. Yin et al. [11] extended graph-based reasoning to automated code review by encoding structural information from code graphs, and Ebrahim and Joy [5] explored semantic-similarity search using neural embeddings for plagiarism detection in academic settings. Earlier surveys [2, 4] document the evolution from string-matching tools (MOSS, JPlag) to PDG-based detectors and, subsequently, to learned representations.

Commonalities. All these works share three premises: they treat code as a structured artifact rather than free text; they evaluate their proposals against curated benchmarks with ground-truth similarity labels; and they report classification metrics (precision, recall, F1) as the primary outcome of interest. Furthermore, the more recent contributions [3, 5, 10, 11] converge on the use of learned representations as superior to purely lexical pipelines, and they all aim at an automated decision rather than human-readable diagnostic output.

Differences. The works diverge along three axes. The first is the underlying representation: Yin et al. [11] and Li et al. [10] use graph neural networks over code graphs, whereas Ebrahim and Joy [5] relies on transformer embeddings over flattened token streams. The second is the application domain: Li et al. [10] target industrial clone detection where false positives are tolerable, while Ebrahim and Joy [5] target academic plagiarism where false

accusations carry substantial cost. The third is the explainability level: graph-based pipelines emit a single similarity score, whereas embedding-based pipelines can produce nearest-neighbor evidence but rarely natural-language justification. Granularity also differs — some methods report a global score [10, 11] while others decompose by region or fragment [5].

Deficiencies and challenges. Three limitations recur across the literature. The first is *training-data dependence*: graph-neural and embedding methods require labeled corpora that are scarce for emerging programming languages and educational settings, limiting transferability [2, 3]. The second is *brittleness to semantic-preserving transformations*: identifier renaming, control-flow refactoring, and data-structure substitutions degrade the performance of token- and AST-based pipelines, a vulnerability that Ebrahim and Joy [5] explicitly call out. The third is the *absence of actionable feedback*: a numeric similarity score is sufficient for forensic plagiarism detection but inadequate for formative educational use, which requires per-dimension explanations and links to underlying causes [1]. These deficiencies set the bar that any educationally oriented detector must clear.

2.2 Generative AI for Software Engineering and Programming Education

LLMs have been applied to code completion, defect prediction, and educational tutoring [12, 19]. Jošt et al. [14] reported gains in student learning outcomes when LLMs supplement programming instruction. Kazemitabaar et al. [13] examined how novices integrate LLM-generated code in CS1 tasks, finding both productivity benefits and risks of shallow learning. Manorat et al. [15] systematically reviewed AI in programming education and identified that the majority of contributions concentrate on *assistance* rather than *evaluation*. Lepp and Kaimre [16] reported mixed effects of generative AI on learning, while Yilmaz and Karaoglan Yilmaz [20] documented improvements in computational-thinking skills attributable to AI tools, and Zvi-Girshin [21] reviewed both benefits and pitfalls of AI tools in introductory programming. Vera Paredes et al. [8] additionally analyzed LLM impact in resource-constrained Latin American contexts.

Commonalities. The reviewed works concur on three points. First, LLMs are technically capable of analyzing code at a level approaching that of human experts. Second, the educational adoption

of LLMs is accelerating but uneven across institutions and curricula. Third, empirical studies consistently find positive effects on *some* learning outcomes (engagement, productivity) while reporting concerns about *others* (depth of understanding, ethical use, academic integrity). All these studies acknowledge that LLM behaviour must be considered alongside pedagogical design rather than in isolation.

Differences. The works adopt different methodological lenses. Kazemitabaar et al. [13] use behavioral observation in self-paced settings, while Jošt et al. [14] run controlled classroom interventions; Manorat et al. [15] adopt a meta-analytic stance. Lepp and Kaimre [16] emphasize student perceptions, in contrast to Yilmaz and Karaoglan Yilmaz [20] who measure cognitive outcomes via standardized instruments. The granularity of LLM use also varies, ranging from passive code generation [13] to active feedback delivery [14]. The choice of LLM differs as well: most studies adopt ChatGPT as a de facto standard, though without justification of that choice.

Deficiencies and challenges. Four challenges emerge from this strand of research. The first is *assistance bias*: as Manorat et al. [15] explicitly note, the literature is heavily skewed toward *generation* use cases, leaving *evaluation* use cases (similarity, quality assessment, efficiency analysis) under-explored. The second is the *lack of model comparability*: most empirical studies use a single LLM (typically ChatGPT) without justification of model choice or comparison against alternatives, limiting the generalizability of findings [14, 16]. The third is the *absence of operational metrics*: studies report educational outcomes but rarely operational characteristics (latency, cost, reliability) needed for institutional deployment [19]. The fourth is *hallucination risk*: Jiang et al. [12] document that LLM outputs can contain factually incorrect or syntactically broken code, a risk that is amplified in evaluation settings where the LLM judgment drives high-stakes decisions about academic integrity or grades.

2.3 Algorithmic Efficiency Analysis and Educational Feedback

Automated estimation of Big O complexity has been studied through static-flow analysis and operation counting [22, 23]. Keuning et al. [7] catalogued common code-quality issues in student programs and showed that automated feedback on efficiency is feasible but rarely integrated with sim-

ilarity detection. Their follow-up systematic review [24] surveyed automated feedback for programming exercises and concluded that few systems address efficiency together with correctness. Malik et al. [6] established that timely feedback on algorithmic performance correlates with measurable learning gains in introductory programming, while Zenil [23] reviewed a broader spectrum of methods for estimating algorithmic complexity from observed program behaviour.

Commonalities. These works share the conviction that automated, immediate, structured feedback improves novice learning, and that algorithmic efficiency is an under-served dimension of feedback compared to syntactic correctness or test-case passing. They also converge on *static* analysis as the practical default, given the cost of dynamic execution at scale. Across the reviewed studies, complexity feedback is consistently framed as a pedagogical lever rather than as a forensic instrument, which differentiates this strand from the similarity-detection literature.

Differences. The granularity of analysis varies considerably across these works. Cormen et al. [22] provide the canonical theoretical framework for asymptotic analysis, expressed in textbooks rather than tools. Zenil [23] review computational methods for algorithmic-complexity *estimation* from output behavior, an empirical alternative to static analysis. Keuning et al. [7] apply pattern matching to identify quality issues, a coarser approach than complexity-class derivation. Malik et al. [6] measure learning gains rather than tool performance, focusing on educational impact rather than tool design. The unit of analysis also varies, ranging from individual constructs (loops, recursion) to whole-program complexity classes.

Deficiencies and challenges. Three persistent deficiencies stand out. The first is *tool fragmentation*: efficiency-analysis tools (e.g., complexity calculators, profilers, linters) are separate artifacts from similarity-detection tools, forcing instructors to combine multiple workflows manually [24]. The second is *limited explainability*: existing complexity-analysis tools emit a Big O class but rarely explain *why*, leaving novices unable to remediate inefficiencies [6]. The third is *restricted language coverage*: most academic tools target a single language (often Java or Python), constraining institutional adoption in multi-language curricula [7, 24]. The conjunction of these deficiencies means that delivering integrated, explainable, and language-flexible efficiency feedback remains

an open problem.

2.4 Prompt Engineering for Structured LLM Tasks

The structured-prompting literature has matured rapidly. Sahoo et al. [25] survey techniques from zero/few-shot prompting to advanced reasoning patterns. Wei et al. [17] introduced Chain-of-Thought (CoT) prompting, achieving 90.2% accuracy on reasoning benchmarks with PaLM-540B. Dhuliawala et al. [18] proposed Chain-of-Verification (CoVe) to mitigate hallucinations through self-verification cycles, reducing factual errors on long-form generation. Liu et al. [26] provide a foundational survey of prompting paradigms across NLP, situating CoT and CoVe within a broader continuum of techniques.

Commonalities. These works share the central insight that LLM behavior can be steered without weight updates by appropriately designed prompts, and that decomposing tasks into reasoning steps (CoT) and verification steps (CoVe) yields measurable accuracy gains. All four works are influential and widely cited, indicating community consensus on their core ideas. They also share an empirical evaluation strategy based on standardized benchmarks, lending comparability to their reported gains.

Differences. CoT [17] addresses reasoning fidelity, while CoVe [18] addresses factual reliability — two distinct failure modes that surface in different downstream tasks. Sahoo et al. [25] provide a taxonomy across techniques and their composition, whereas Liu et al. [26] historicize the evolution from fine-tuning to prompting. The empirical evaluations also differ: CoT was validated on multi-step arithmetic and commonsense benchmarks, whereas CoVe targeted long-form factual generation. Neither original work targeted code-analysis tasks specifically.

Deficiencies and challenges. Two limitations matter for code-analysis applications. The first is *domain transfer*: CoT and CoVe were validated on natural-language reasoning benchmarks; their effectiveness on structured-output tasks over source code (multi-dimensional scoring with embedded justifications) has not been independently evaluated [25]. The second is *composition*: there is no published prompt that operationalizes CoT and CoVe *jointly* for a code-comparison task that requires both step-by-step reasoning across dimensions and self-verification of score-justification consistency. Existing systems either invoke CoT

alone for reasoning tasks or invoke CoVe alone for factuality tasks, but not both in a unified workflow.

2.5 Synthesis and Comparative Advantages of SimilCode

The reviewed literature evidences three converging trends. First, learned and LLM-based representations are displacing syntactic similarity baselines but remain dependent on training data and brittle to semantic-preserving transformations. Second, LLMs exhibit semantic reasoning suitable for code comparison, yet evaluation use cases are under-represented compared to assistance use cases. Third, educational settings demand combined similarity *and* efficiency feedback, a combination not addressed by any single existing tool. Table 1 positions SimilCode against representative reviewed works along seven analytical dimensions.

The additional advantages of SimilCode over the reviewed works can be summarized along six lines, grouped here into three thematic clusters. The first cluster concerns *integration*. SimilCode delivers *dual-task integration*: no reviewed work combines similarity detection and algorithmic-efficiency analysis in a single tool, and existing pipelines force instructors to orchestrate separate artifacts manually [24]. By unifying both tasks under one prompt and one user interface, the system removes the workflow fragmentation identified in Section 2.3. It also offers *multidimensional similarity*: where reviewed pipelines emit a single similarity score, SimilCode reports five orthogonal dimensions (lexical, structural, stylistic, functional, syntactic) with per-dimension justifications. This granularity is required for formative feedback that token-based and embedding-based methods cannot offer [1, 5], and it answers the actionability deficiency identified in Section 2.1.

The second cluster concerns *empirical grounding*. SimilCode provides *comparative LLM benchmarking*: most LLM-in-education studies fix on a single model without justification [13, 14, 16], and this work delivers the first published systematic comparison of four leading commercial LLMs under a controlled protocol, addressing the model-comparability deficiency identified in Section 2.2. It also reports *operational metrics* (response time, justification quality) beyond the educational outcomes prevalent in prior studies, supplying precisely the operational evidence that Viet and Vinh [19] flag as missing for institutional deployment.

The third cluster concerns *methodological ro-*

bustness. SimilCode exhibits *transformation robustness* because the analysis is delegated to a semantic-aware LLM rather than to a token-level comparator; this design renders it resilient to identifier renaming and control-flow refactoring that defeat AST/PDG pipelines [5], addressing the brittleness deficiency of Section 2.1. The system further introduces a *novel CoT+CoVe composition*: the standardized prompt is, to our knowledge, the first published artifact that jointly operationalizes Chain-of-Thought reasoning and Chain-of-Verification self-checking for multi-dimensional code-similarity scoring, addressing the composition deficiency identified in Section 2.4. Collectively, these six advantages reposition the contribution from an incremental tool to an integrated educational-evaluation system grounded in empirical model selection.

3 Methodology

3.1 Research Design

We adopted a mixed applied-experimental design. The independent variable was the LLM (four levels: Claude Opus 4.1, GPT-5, Gemini 2.5 Pro, DeepSeek-V3). Dependent variables were classification precision, response time (s), and justification quality (Likert-derived composite). Control variables comprised programming language (C#, Java), code complexity (lines of code, number of methods), and execution environment. The design used repeated measures: every test pair was processed by all four models under identical conditions, enabling paired comparisons.

3.2 Dataset Construction

We curated 120 code pairs (60 in C#, 60 in Java) distributed equally across four similarity categories: *identical* (100%), *functional* (70–90%), *structural* (40–60%), and *different* (0–30%). Each pair was constructed from canonical introductory-programming problems (sorting, searching, recursion, data-structure manipulation) and validated by two independent reviewers using a code-walkthrough protocol. Pairs were stratified by length (15–80 LOC) and method count (1–6) to ensure balanced complexity. Ground-truth labels were assigned a priori through manual code analysis: identical pairs contained byte-identical implementations; functional pairs preserved input/output behavior with refactored control flow; structural pairs shared algorithmic skeleton but

Table 1: Comparative positioning of SimilCode against representative reviewed works. Legend: ✓ = supported; — = not supported; N/A = not applicable. Sim. detect. = similarity detection; Effic. analysis = efficiency analysis; Multi-dim. sim. = multi-dimensional similarity; Multi-LLM bench. = multi-LLM benchmarking; NL justif. = natural-language justifications.

Work	Sim. detect.	Effic. analysis	Multi-dim. sim.	Multi-LLM bench.	NL justif.	Multi-lang. support	Domain
Li et al. [10]	✓	—	—	—	—	limited	industrial
Yin et al. [11]	✓	—	—	—	—	limited	code review
Ebrahim and Joy [5]	✓	—	—	—	partial	limited	academic
Zhang and Saber [3]	survey	—	—	—	—	—	generic
Kazemitabaar et al. [13]	—	—	—	—	✓	single	assistance
Jošt et al. [14]	—	—	—	—	✓	single	assistance
Keuning et al. [7]	—	✓	—	N/A	partial	single	feedback
Malik et al. [6]	—	✓	—	N/A	—	single	learning
SimilCode (this work)	✓	✓	✓(5d)	✓(4)	✓	C#, Java ⁺	education

⁺Extensible to additional languages via the provider-agnostic adapter.

differed in semantics; different pairs solved unrelated problems. Inter-rater agreement on category assignment exceeded 95%; disagreements were resolved by consensus.

3.3 Standardized Prompt

A single prompt template was designed using CoT [17] and CoVe [18]. The prompt instructs the model to (i) reason step-by-step over five similarity dimensions (lexical, structural, stylistic, functional, syntactic), (ii) self-verify internal coherence between numeric scores and textual justifications, and (iii) emit a structured response. Output fields are five integer scores (0–100), one global score, and per-dimension justifications. The template is parameterized only by the two code fragments and the language identifier; all other content is invariant across models, controlling for prompt-formulation variance.

3.4 Experimental Procedure

Each of the 120 pairs was submitted to each of the four models through their official chat interfaces, yielding 480 total trials. The execution environment was held constant: Intel Core i5-10400 CPU, 16 GB RAM, Windows 11 Pro, mean network throughput 94.92 Mbps. Response time was measured from prompt submission to full-response receipt using a digital chronometer with 0.01 s resolution. Models were processed sequentially (all 120 pairs per model) to limit temporal coupling between providers. Outputs were stored verbatim for downstream analysis.

Table 2: Justification Quality Rubric

Dimension	Item	Scale
Clarity	Comprehensible explanation	1–5
	Correct technical terminology	1–5
	Logical structure	1–5
	Lexical coverage	yes/no
Completeness	Structural coverage	yes/no
	Functional coverage	yes/no
	Stylistic coverage	yes/no
	Score–justification consistency	1–5
Coherence	Internal logical consistency	1–5

3.5 Quality Assessment Rubric

Justification quality was scored across three dimensions on the rubric in Table 2: *clarity* (three 1–5 ordinal items), *completeness* (four binary coverage items), and *coherence* (two 1–5 ordinal items). Two trained evaluators independently scored all 480 justifications; Cohen’s $\kappa = 0.81$ indicated substantial agreement. Disagreements were resolved by discussion. A model-level composite score was computed as the mean of normalized item scores.

3.6 Statistical Analysis

For precision and response time, normality (Shapiro–Wilk) and homoscedasticity (Levene) were verified. One-way ANOVA tested between-model differences ($\alpha = 0.05$), followed by Tukey HSD post-hoc when significant. Justification quality, an ordinal composite, was analyzed with the non-parametric Kruskal–Wallis test followed by Dunn’s pairwise comparisons with Bonferroni correction. Effect sizes were reported as η^2 for

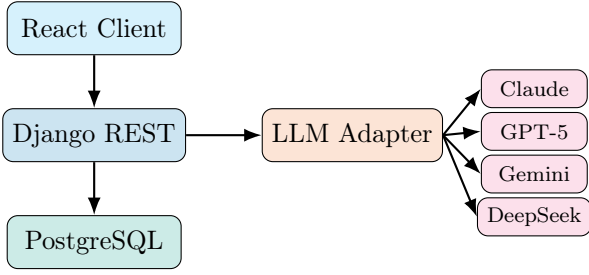


Figure 1: SimilCode three-tier architecture with provider-agnostic LLM adapter.

ANOVA and ϵ^2 for Kruskal–Wallis. Analyses were implemented in Python 3.14 using NumPy, Pandas, and SciPy.

3.7 System Engineering Methodology

Architecture. SimilCode follows a three-tier client–server architecture: a React 18.2 single-page front end, a Django 4.2 REST back end, and a PostgreSQL 15 persistence layer. LLM access is brokered through a provider-agnostic adapter that normalizes responses across Anthropic, OpenAI, Google, and DeepSeek APIs. Authentication uses JWT; API credentials are encrypted at rest. Figure 1 presents the deployment view.

Development methodology. Given the bounded scope and well-defined requirements, we adopted rapid prototyping [27, 28]. Requirements were elicited through semi-structured interviews with five programming instructors (mean experience 8.4 years). The resulting specification follows ISO/IEC/IEEE 29148:2018 [29] and comprises 22 functional requirements grouped into five modules (individual comparison, group comparison, AI integration, language management, comparison administration), 9 non-functional requirements (performance, security, scalability), and 8 security/privacy requirements.

Modules. The *individual comparison* module accepts two code fragments, dispatches them to the selected LLM with the standardized prompt, parses the response, computes Big O time and space complexity through static-flow inspection, and renders an integrated report. The *group comparison* module generalizes to $n \geq 3$ fragments, computing the $\binom{n}{2}$ pairwise similarities and ranking efficiency. Big O analysis is implemented as a hybrid: AST-derived loop-nesting depth and operation counting yield a candidate complexity, which the LLM validates and refines using its semantic understanding.

3.8 Expert Validation

After the comparative experiment and system implementation, five domain experts (instructors of programming courses, mean experience 6.8 years) participated in individual 45-minute sessions. Each session comprised a 10-minute system demonstration, 20 minutes of hands-on interaction with 10 representative code pairs, and a 15-minute structured questionnaire. The instrument comprised 18 Likert-scale items (1–5) across three sections — practical usefulness (7 items), result accuracy (6 items), justification interpretability (5 items) — plus two open-ended items. Sample size of five aligns with established usability-evaluation guidance [30], which finds that 3–5 evaluators suffice to surface most usability issues.

3.9 Ethical Considerations

The study followed standard research-ethics norms [31, 32]: informed consent was obtained from all participants, data were anonymized and reported in aggregate, LLM API usage complied with provider terms, and all evaluation code was either author-authored or in the public domain. Raw experimental data are archived for replicability.

4 Results

4.1 Precision of Detection

Across the 480 trials, all four models classified 118 of 120 cases correctly, yielding identical mean precision of 98.3%. Disaggregated by language: Claude Opus 4.1 and Gemini 2.5 Pro achieved 100% in both C# and Java; GPT-5 and DeepSeek-V3 reached 96.7% in C# (two structural-vs-functional confusions) and 100% in Java. The one-way ANOVA returned $F(3, 476) = 0.000$, $p = 1.000$, $\eta^2 < 0.001$, indicating no statistically detectable between-model variance. We therefore retain $H_{0,1}$: precision is equivalent across models. Figure 2 visualizes the result.

4.2 Response Time

Mean response times (s) and dispersion are reported in Table 3. Claude Opus 4.1 was fastest at 19.08 s overall, followed by DeepSeek-V3 (23.15 s), GPT-5 (24.72 s), and Gemini 2.5 Pro (25.73 s). The fastest-to-slowest gap of 6.65 s represents a 34.8% relative latency increase. ANOVA returned $F(3, 476) = 30.89$, $p < 0.001$, $\eta^2 = 0.163$ (large

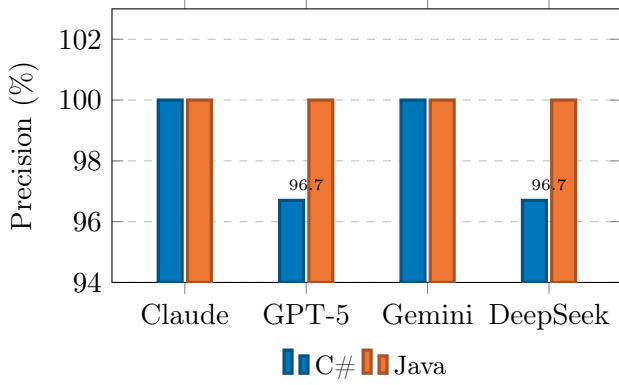


Figure 2: Classification precision per model and language ($n = 60$ per cell). Bars at the top of the panel reach 100%; the 96.7% values for GPT-5 and DeepSeek-V3 in C# are labelled. ANOVA yields $p = 1.000$.

Table 3: Response Time Statistics by Model (Seconds)

Model	Mean	SD	CV (%)	Range
Claude Opus 4.1	19.08	2.41	12.6	14.2–26.1
DeepSeek-V3	23.15	3.88	16.8	16.7–32.9
GPT-5	24.72	3.62	14.6	17.4–34.0
Gemini 2.5 Pro	25.73	4.11	16.0	18.0–35.6

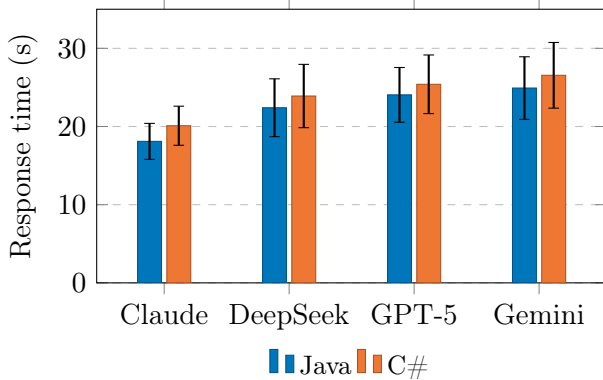


Figure 3: Mean response time per model and language with ± 1 SD error bars (numerical values in Table 3). Claude Opus 4.1 is significantly faster than all alternatives (Tukey HSD $p < 0.001$).

effect). Tukey HSD post-hoc indicated Claude Opus 4.1 differed significantly from each of the other three models ($p < 0.001$ in all three pairwise contrasts), while differences among GPT-5, Gemini, and DeepSeek were not all significant. We reject $H_{0,2}$ in favor of $H_{1,2}$.

Table 4: Justification Quality by Dimension and Model (Mean, 1–5 Scale)

Dimension	Claude	GPT-5	Gemini	DeepSeek
Clarity	4.97	4.86	4.93	4.81
Completeness	4.95	4.84	4.90	4.82
Coherence	4.96	4.85	4.93	4.86
Composite	4.96	4.85	4.92	4.83

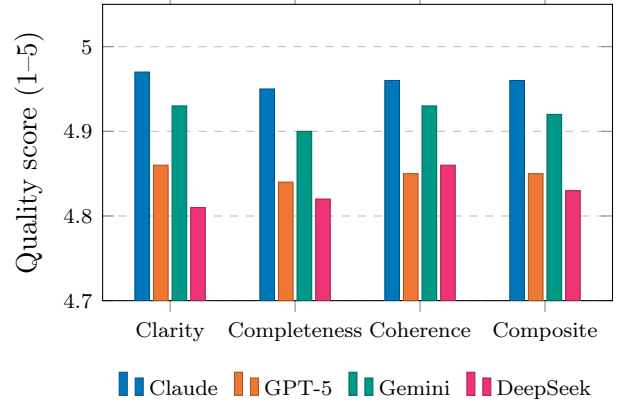


Figure 4: Justification quality across rubric dimensions (numerical values in Table 4). Claude Opus 4.1 leads in all dimensions; differences are significant against DeepSeek and GPT-5.

4.3 Justification Quality

Composite quality scores (range 0–5) are summarized in Table 4. Claude Opus 4.1 obtained the highest mean (4.96), followed by Gemini 2.5 Pro (4.92), GPT-5 (4.85), and DeepSeek-V3 (4.83). Kruskal–Wallis returned $H = 11.50$, $p = 0.009$, $\epsilon^2 = 0.024$. Dunn’s post-hoc with Bonferroni correction indicated Claude differed significantly from DeepSeek-V3 ($p = 0.012$) and from GPT-5 ($p = 0.041$); other contrasts were non-significant. We reject $H_{0,3}$ in favor of $H_{1,3}$.

4.4 Consolidated Hypothesis Tests

Table 5 summarizes the inferential results. Two of three specific hypotheses are rejected, supporting the general conclusion that performance differences between LLMs are detectable, with Claude Opus 4.1 dominating on operational dimensions despite precision parity.

4.5 System Functional Output

Figure 5 schematically illustrates the integrated output of the individual-comparison module: a five-dimensional similarity profile plus per-fragment Big O classification. For an illustra-

Table 5: Summary of Hypothesis Tests

Hypothesis	Test	Statistic	p	Decision
$H_{0,1}$ Precision	ANOVA	$F = 0.00$	1.000	Retain
$H_{0,2}$ Time	ANOVA	$F = 30.89$	$< .001$	Reject
$H_{0,3}$ Quality	Kruskal–Wallis	$H = 11.50$	0.009	Reject

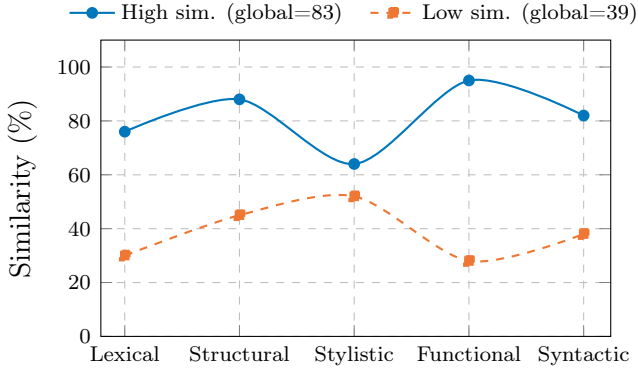


Figure 5: Five-dimensional similarity profile output by the SimilCode individual-comparison module for two illustrative pairs.

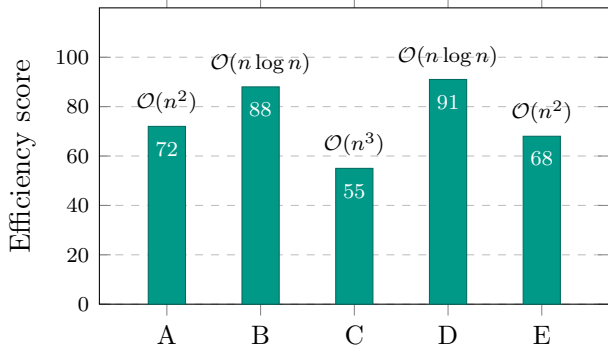


Figure 6: Group-comparison module: efficiency ranking for a 5-fragment cohort with annotated Big O complexities (above each bar) and efficiency scores (inside bar tops). Fragment D leads.

tive pair of bubble-sort variants, the system reported lexical 76, structural 88, stylistic 64, functional 95, syntactic 82, global 83 — and identified both fragments as $\mathcal{O}(n^2)$ time / $\mathcal{O}(1)$ space, with fragment B marginally more efficient due to the absence of a redundant inner condition.

4.6 Expert Validation

The five experts produced a global mean of 4.60/5.00 (Table 6). Practical-usefulness items obtained the highest section mean (4.80), driven by perfect or near-perfect scores on similarity detection, Big O usefulness, and intent-to-adopt. Result-accuracy averaged 4.60, with the identical-

Table 6: Expert Validation Results (1–5 Likert)

Expert	Usefulness	Accuracy	Interpret.	Mean
E1	5	5	4	4.75
E2	5	4	5	4.50
E3	4	5	5	4.75
E4	5	4	4	4.25
E5	5	5	5	4.75
Mean	4.80	4.60	4.60	4.60

pair detection item reaching 5.00 across all experts. Justification-interpretability averaged 4.60, with the score-explanation item also at 5.00. Open-ended feedback emphasized the value of integrating efficiency and similarity in a single screen and requested support for additional languages.

5 Discussion

The findings refine and extend the literature in three directions. The first direction concerns precision parity. The equivalent precision observed across four leading commercial LLMs aligns with the maturation trend documented by Zhang and Saber [3] for ML-based similarity detection, and extends it to general-purpose foundation models that lack code-specific training. Practically, this implies that institutions should not select among modern LLMs on precision grounds alone — a result that simplifies procurement but reframes selection toward operational metrics.

The 6.65 s latency gap between Claude Opus 4.1 and Gemini 2.5 Pro becomes economically meaningful at institutional scale. In a course of 200 students with 10 weekly submissions, the cumulative delta exceeds 3.7 hours per week, validating Francinton et al. [4]’s emphasis on temporal scalability. This finding suggests that response time, often treated as a secondary concern in educational AI research, should be elevated to a primary selection criterion when LLMs are deployed at scale.

The second direction concerns the quality of LLM-generated explanations. The differential observed in our experiments aligns with Keuning et al. [24]’s finding that automated-feedback use-

fulness depends critically on clarity. Our composite quality differences are numerically small but statistically significant. Qualitative inspection of justifications further indicates that the differences manifest in technical specificity: Claude tends to cite concrete control-flow constructs, while DeepSeek occasionally produces generic phrasings. For educational deployment — where justifications are directly read by students — this margin is non-trivial.

The CoT/CoVe-engineered prompt likely contributes to the high absolute scores observed for all models, consistent with Wei et al. [17] and Dhuliawala et al. [18]. This suggests that prompt engineering may compress the quality gap between LLMs even when the models differ substantially in underlying capabilities, although the residual gap remains statistically detectable.

The third direction concerns the integration of similarity and efficiency analysis. The unified workflow proposed in this study addresses a gap identified across multiple reviews [1, 2, 21]. Existing tools either detect plagiarism (MOSS, JPlag) or analyze quality (PMD, SonarQube), but not both. The expert-validation result (4.80/5 on usefulness, with 5/5 on Big O usefulness) provides empirical support for this design decision, complementing Manorat et al. [15]’s observation that AI-in-education research over-emphasizes assistance over assessment.

Limitations. Three limitations bound the generalizability of these findings. The first is dataset coverage: the experimental corpus is restricted to C# and Java, and results may differ for languages with distinct paradigms (Python’s dynamic typing, C++’s template metaprogramming). The second is model evolution: LLMs evolve rapidly, and the specific model versions tested in 2025 may be superseded; we mitigate this via the provider-agnostic adapter, which permits straightforward re-benchmarking. The third is sample size in expert validation: $n = 5$ aligns with usability heuristics [30] but does not capture institutional-scale adoption dynamics, and future work should include longitudinal studies in real classrooms.

Implications. The system architecture and evaluation protocol are reusable beyond the immediate study. The standardized prompt and rubric can serve as benchmarks for future LLMs, and the adapter pattern permits incremental integration of new providers. For research on AI-augmented programming education, the results motivate a shift from reporting precision in isolation to multidimensional evaluation that prioritizes latency and explanation quality.

mensional evaluation that prioritizes latency and explanation quality.

6 Conclusions

This work contributes empirical, methodological, and engineering advances to the intersection of LLMs and software-engineering education. Empirically, four state-of-the-art commercial LLMs achieve equivalent precision (98.3%) on multidimensional code-similarity detection. They differ significantly in latency (Claude Opus 4.1 fastest at 19.08 s) and in justification quality (Claude Opus 4.1 highest at 4.96/5). Methodologically, the CoT/CoVe-engineered prompt and three-axis rubric provide a reproducible benchmark for future LLM evaluations on code-comparison tasks.

As an engineering artifact, SimilCode integrates five-dimensional semantic similarity with automated Big O efficiency analysis in a single deployable web application. It addresses a documented gap in academic code-evaluation tooling and was validated by expert instructors at 4.60/5.00.

Future work will pursue four directions. First, evaluation will be extended to Python, JavaScript, and C++ to assess language-paradigm effects on LLM performance. Second, a public benchmark with verified-complexity reference algorithms will be established for systematic Big O validation. Third, longitudinal classroom studies will measure learning outcomes attributable to the system. Fourth, a continuous-evaluation protocol will be implemented to update LLM rankings as new model versions are released.

Acknowledgments

The authors thank the five expert instructors who contributed to system validation, and the Faculty of Computer Sciences of Universidad Técnica Estatal de Quevedo for institutional support.

Conflict of Interest

The authors declare no conflict of interest.

References

- [1] Gunwoo Lee, Jindae Kim, Myung-seok Choi, Rae-Young Jang, and Ryong Lee. Review of code similarity and plagiarism detection research studies. *Applied Sciences*, 13(20): 11358, 2023. doi: 10.3390/app132011358.

- [2] Morteza Zakeri-Nasrabadi, Saeed Parsa, Mohammad Ramezani, Chanchal Roy, and Masoud Ekhtiarzadeh. A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges. *Journal of Systems and Software*, 204:111796, 2023. doi: 10.1016/j.jss.2023.111796.
- [3] Zixian Zhang and Takfarinas Saber. Machine learning approaches to code similarity measurement: A systematic review. *IEEE Access*, 13:51729–51764, 2025. doi: 10.1109/ACCESS.2025.3553392.
- [4] Reggi Francinton, Oscar Karnalim, and Mewati Ayub. A scalable code similarity detection with online architecture and focused comparison for maintaining academic integrity in programming. *International Journal of Online and Biomedical Engineering (iJOE)*, 16(10):40–52, 2020. doi: 10.3991/ijoe.v16i10.14289.
- [5] Fahad Ebrahim and Mike Joy. Semantic similarity search for source code plagiarism detection: An exploratory study. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*, pages 360–366. ACM, 2024. doi: 10.1145/3649217.3653622.
- [6] Sohail Iqbal Malik, Roy Mathew, Abir Al-Sideiri, Jasiya Jabbar, Rim Al-Nuaimi, and Ragad M. Tawafak. Enhancing problem-solving skills of novice programmers in an introductory programming course. *Computer Applications in Engineering Education*, 30(1): 174–194, 2022. doi: 10.1002/cae.22450.
- [7] Hieke Keuning, Bastiaan Heeren, and Johan Jeuring. Code quality issues in student programs. In *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE)*, pages 110–115. ACM, 2017. doi: 10.1145/3059009.3059061.
- [8] Diana A. Vera Paredes, Otto O. Franco Arias, and Luis C. Córdova Martínez. Impacto de la inteligencia artificial en el aprendizaje de la programación informática en los estudiantes universitarios. *Ciencia y Educación*, 6(3):33–50, 2025. doi: 10.5281/zenodo.15056104.
- [9] Jean Gabriel Guerrero-Dib, Luis Portales, and Yolanda Heredia-Escorza. Impact of academic integrity on workplace ethical behaviour. *International Journal for Educational Integrity*, 16(1):1–18, 2020. doi: 10.1007/s40979-020-0051-3.
- [10] Zhenhao Li, Hang Lei, Zhichao Ma, and Fengyun Zhang. Code similarity prediction model for industrial management features based on graph neural networks. *Entropy*, 26(6):505, 2024. doi: 10.3390/e26060505.
- [11] Ying Yin, Yuhai Zhao, Yiming Sun, and Chen Chen. Automatic code review by learning the structure information of code graph. *Sensors*, 23(5):2551, 2023. doi: 10.3390/s23052551.
- [12] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*, 2024. doi: 10.48550/arXiv.2406.00515. arXiv:2406.00515.
- [13] Majeed Kazemitabaar, Xinying Hou, Austin Z. Henley, Barbara J. Ericson, David Weintrop, and Tovi Grossman. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. ACM, 2023. doi: 10.1145/3631802.3631806.
- [14] Gregor Jošt, Viktor Taneski, and Sāšo Karakatič. The impact of large language models on programming education and student learning outcomes. *Applied Sciences*, 14(10): 4115, 2024. doi: 10.3390/app14104115.
- [15] Pisut Manorat, Suppawong Tuarob, and Siripen Pongpaichet. Artificial intelligence in computer programming education: A systematic literature review. *Computers and Education: Artificial Intelligence*, 8:100403, 2025. doi: 10.1016/j.caeai.2025.100403.
- [16] Marina Lepp and Joosep Kaimre. Does generative AI help in learning programming: Students’ perceptions, reported use and relation to performance. *Computers in Human Behavior Reports*, 18:100642, 2025. doi: 10.1016/j.chbr.2025.100642.
- [17] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large

- language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 2022.
- [18] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3563–3578, 2024. doi: 10.18653/v1/2024.findings-acl.212.
- [19] Nguyen Van Viet and Nguyen The Vinh. Large language models in software engineering: A systematic review and vision. *Journal of Education for Sustainable Innovation*, 2(2): 146–156, 2024. doi: 10.56916/jesi.v2i2.968.
- [20] Ramazan Yilmaz and Fatma Gizem Karaoglan Yilmaz. The effect of generative artificial intelligence (AI)-based tool use on students’ computational thinking skills, programming self-efficacy and motivation. *Computers and Education: Artificial Intelligence*, 4:100147, 2023. doi: 10.1016/j.caeai.2023.100147.
- [21] Rina Zvieli-Girshin. The good and bad of AI tools in novice programming education. *Education Sciences*, 14(10):1089, 2024. doi: 10.3390/educsci14101089.
- [22] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, 4th edition, 2022.
- [23] Hector Zenil. A review of methods for estimating algorithmic complexity: Options, challenges, and new directions. *Entropy*, 22(6):612, 2020. doi: 10.3390/e22060612.
- [24] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1):1–43, 2018. doi: 10.1145/3231711.
- [25] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024. doi: 10.48550/arXiv.2402.07927.
- [26] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023. doi: 10.1145/3560815.
- [27] Steven D. Tripp and Barbara Bichelmeyer. Rapid prototyping: An alternative instructional design strategy. *Educational Technology Research and Development*, 38(1):31–44, 1990. doi: 10.1007/BF02298246.
- [28] Shuanglong Chang, Juntao Gao, and Weiru Wang. Automatic generation of software prototype data for rapid requirements validation. *Electronics*, 14(17):3497, 2025. doi: 10.3390/electronics14173497.
- [29] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 – systems and software engineering — life cycle processes — requirements engineering. International Standard, 2018.
- [30] Imam Kusmaryono, Dyana Wijayanti, and Hevy Risqi Maharani. Number of response options, reliability, validity, and potential bias in the use of the Likert scale education and social science research: A literature review. *International Journal of Educational Methodology*, 8(4):625–637, 2022. doi: 10.12973/ijem.8.4.625.
- [31] Liliana Rodrigues. Ethical procedures in scientific research. *Journal of Education, Society and Behavioural Science*, 36(7):78–84, 2023. doi: 10.9734/jesbs/2023/v36i71238.
- [32] David B. Resnik and Mohammad Hosseini. The ethics of using artificial intelligence in scientific research: New guidance needed for a new tool. *AI and Ethics*, 5(2):1499–1521, 2024. doi: 10.1007/s43681-024-00493-8.